VIEWS

Delivery Note

Nine Deep-Learning Forecast Models

— ◇ —

Identifier	`r2darts2-pgm-global-calibration-2026-10`
Checksum	`60597598`
Date	10-10-2026
Author	Simon Polichinel von der Maase

Scope *what the data is*

Partition	Calibration
Spatial resolution	PRIO-GRID cell
Spatial extent	Global land — 64,818 cells
Temporal resolution	Month
Temporal extent	Forecasts Jan 2018 to Dec 2021; trained on Jan 1990 to Dec 2017
Forecast structure	13 rolling origins, each forecasting 36 months
Targets	State-based, non-state and one-sided fatalities

Provenance *what produced it*

Model library	views-r2darts2 0.2.4, wrapping Darts (`unit8co/darts`)
Framework	views-pipeline-core 3.3.4
Data backend	views-datafactory 1.13.0
Source commit	`views-models@be7b7b6`
Computed on	Rented cloud GPUs (RunPod)

PRIO – Oslo

VIEWS
PREDICTING CONFLICT & HUMANITARIAN IMPACTS

PRIO

1 Context

This Delivery Note accompanies `r2darts2-pgm-global-calibration-2026-10`, a set of conflict forecasts produced by nine deep-learning models on the calibration partition at global PRIO-GRID month. It is the authoritative description of what was handed over: the data itself carries no documentation, and the folder is not self-explanatory.

The purpose of this note is to:

- record what was delivered, in enough detail to identify it again later;
- describe the models that produced it, and what distinguishes them from one another;
- state how the files are organised and how they are intended to be consumed; and
- set out what is *not* in the delivery, and what is not yet known about it.

This note covers:

- the nine models, their architectures, and the published work they come from (Sections 2–3);
- the transformations applied to the data and undone again (Section 4);
- the objective functions they were trained on, including one written in-house (Section 5);
- the file layout, schema and units of the delivered forecasts (Section 6); and
- what the delivery excludes, and the caveats that bear on using it (Sections 7–8).

This note explicitly does *not* cover:

- **any evaluation of the models.** Nothing here has been scored, ranked or compared. No metric has been computed beyond a check that the files are not corrupt;
- any recommendation as to which models to prefer, or whether to use them at all;
- the validation or forecasting partitions, or any live forecast; and
- the internal machinery that produced the run — what it took to compute these forecasts is recorded separately, in `reports/postmortem_darts_calibration_2026-10.md`.

The delivery is a fixed artefact. It is identified by the identifier and checksum on the title page — quote both — and is not revised in place: any correction is issued as a new delivery with a new identifier and a new checksum.

2 The Models

All nine are **off-the-shelf deep-learning models for time series**, taken from the Python library **Darts** (`unit8co/darts`). The architectures are the library's, unmodified — each is a published model used as Darts implements it. What we supply is the data, the configuration, and for six of the nine a loss function of our own (Section 5).

They are *global* models in the forecasting sense: a single set of parameters is pooled across all 64,818 cells. The closest familiar analogue is a pooled panel estimator — a cell with almost no conflict history is still forecast using structure learned from the whole grid. Each cell's own level and scale are restored separately, by the normalisation described in Section 4.

Table 1: The nine delivered models.

Model	Type	Loss	Predictors	Hyperparameters	Features
dark_river	N-BEATS	MSE	Outcomes only	link	link
old_rules	N-BEATS	Spotlight	Outcomes only	link	link
blue_ocean	N-BEATS	Spotlight	+ 69 covariates	link	link
silent_fox	TSMixer	MSE	Outcomes only	link	link
dancing_monkey	TSMixer	Spotlight	Outcomes only	link	link
brave_heart	TSMixer	Spotlight	+ 69 covariates	link	link
red_hawk	TiDE	MSE	Outcomes only	link	link
dark_necessities	TiDE	Spotlight	Outcomes only	link	link
golden_eagle	TiDE	Spotlight	+ 69 covariates	link	link

The link columns open each model's configuration files on GitHub, fixed at the delivery commit on the title page: **Hyperparameters** is `config_hyperparameters.py`, which sets the architecture, the loss and the training schedule; **Features** is `config_queryset.py`, which declares the predictors the model was given.

The names are arbitrary labels, assigned according to the `adjective_noun` convention used across views-models, and carry no meaning.

Within each block of three, exactly two settings vary — the loss function, and whether the model sees covariates. Everything else is identical: layer widths, depth, training epochs, learning rate, early stopping. So inside a block, each row differs from the one above it in a single setting. Across blocks the architecture changes and the comparison is a different one.

- **Outcomes only** — the model's inputs are the past values of the three outcomes and nothing else. Purely autoregressive. Six models: the first two rows of each block.
- **+ 69 covariates** — the same three histories plus 69 predictors: ACLED event and fatality counts, land cover shares, population and built-up area, infant and child mortality, V-Dem governance indices, SHDI development indices, travel time to cities, mineral and petroleum deposits, growing season and harvested area, and terrain. Three models: the third row of each block.

Note that the two splits are not the same split. Six models are outcomes-only and six use the Spotlight loss, but they are different sixes: `blue_ocean`, `brave_heart` and `golden_eagle` have covariates *and* Spotlight, while `dark_river`, `silent_fox` and `red_hawk` have neither.

There is no model combining MSE with the covariates. Three configurations exist, not four.

Covariate Preparation

Covariates are prepared in two stages, by different parties and to different standards.

Within the datafactory, preparation is specified. Annual sources are extended across their twelve months by per-source rules; values outside a source's observed range are manufactured, zero before the first observation and the last value held after it; absent coverage is filled with zeros rather than nulls; raster and polygon sources are attached to grid cells by aggregation and spatial join. Feature engineering is excluded by design, cross-feature derivation being assigned to the model layer. Each of these is a recorded decision with an architecture decision record behind it — ADR-014, ADR-042, ADR-047 and ADR-052 in the datafactory repository, with a documentation card per source. That is the authoritative account and this note does not restate it.

Preparation after the fetch is set in the model configuration, and is documented only by the configuration itself. Two operations apply here: the selection of which covariates each model receives, and the transformations described in Section 4. Both are visible in the files linked in Table 1. Neither carries a decision record or a written rationale, and whether either is warranted is not established by this note.

Covariate Coverage

The datafactory publishes, for each source, the month range in which its observations are real. Values outside that range exist but are manufactured. Checked against this delivery's partition, four of the five sources cover it fully and one does not.

Table 2: Source coverage against the calibration partition.

Source	**Observed from**	**Training 1990–2017**	**Test 2018–2021**
ACLED (7 covariates)	January 2020	**none — all zeros**	real from January 2020
V-Dem	January 1989	real	real
SHDI	January 1989	real	real
Population	January 1989	real	real
Built-up surface	January 1989	real	real

The seven ACLED covariates are therefore zero for every month these models were fitted on, and become non-zero partway through the forecast window. This is a sharper problem than an absent feature: a predictor that is constant in training and varies in test is one the model has learned to disregard. Nor is it visible in the data, since manufactured and observed zeros are written identically.

The coverage bounds are published so that manufactured values can be identified, and were available when these models were configured; the omission is downstream of the datafactory, not within it. Three models are affected — `blue_ocean`, `brave_heart` and `golden_eagle`. The six outcomes-only models use no covariates.

3 Model Architectures

All three are feed-forward neural networks: information passes once from input to output, with no internal memory carried between time steps. They differ from one another in how the layers are arranged, described below.

N-BEATS

A deep stack of fully connected layers with backward and forward residual links. It builds the forecast as a sum of learned basis functions, which is what makes it partly interpretable. Designed for univariate point forecasting and benchmarked on the M3, M4 and TOURISM competition datasets.

Implemented in Darts as `NBEATSModel`; see Oreshkin et al. (2020).

TSMixer

Dense layers only, applied alternately along the time axis and across the predictor variables, so the model mixes temporal structure and cross-variable structure in turn. Built for the multivariate case.

Implemented in Darts as `TSMixerModel`; see Chen et al. (2023).

TiDE

An encoder–decoder, again dense layers only. It compresses the history and the predictors into a single code, then decodes the whole 36-month path from it in one pass.

Implemented in Darts as `TiDEModel`; see Das et al. (2023).

4 Transformations

Two transformations are applied between the data and the model, and both are reversible. No others are used: the configurations declare no logarithm, no min–max or standard scaling, no winsorising and no clipping of values.

An `asinh` transform. All nine models apply it to the three outcomes; the three covariate-using models apply it to their inputs as well. The inverse hyperbolic sine is used rather than a logarithm because it is defined at zero, which matters at this resolution — **99.6%** of observed cell-months in this window are exactly zero.

Reversible instance normalisation. Each cell's series is standardised before the network sees it and the standardisation is undone on the way out. This is what returns a cell's own level and scale after the pooling described in Section 2 (Kim et al., 2022).

Both inversions happen before the delivered files are written, so **every column in this delivery is intended to be on the original count scale** and no back-transformation should be applied to it.

That is an intention supported by a check rather than a proof. The largest delivered value is a few hundred fatalities, which is plausible as a count and absurd on the transformed scale, and the totals are the same order of magnitude as the observed field. Nothing in the delivery verifies the inversion formally.

Implications for Interpretation

Whether `asinh` is a prudent choice here, and what follows from it, are questions this note does not answer and is not responsible for answering. Two are worth stating because they bear on how the delivered numbers should be read.

Inverting a point estimate is biased. A model fitted to minimise error on a transformed scale estimates the transformed quantity, and applying a nonlinear inverse to that estimate does not recover the conditional mean — long settled, and not specific to `asinh` (Duan, 1983). Collapsing draws in count space avoids it, but there are no draws here: all nine models produce a single value per cell, so the inverse is necessarily applied to a point. The inverse of `asinh` is convex over the positive range, so the bias is downward. Its magnitude for these models is unquantified, and whether the delivered totals sit below observed totals is measurable without refitting.

The transform is under active criticism. Elasticities estimated from inverse-hyperbolic-sine-transformed regressions depend on the units of the dependent variable (Bellemare and Wichman, 2020), and where there is a mass at zero the magnitude of an estimated effect on any log-like transform can be altered arbitrarily by rescaling the outcome (Chen and Roth, 2024). That literature concerns the interpretation of coefficients in causal inference rather than prediction followed by inversion, so whether it carries over here is an open question rather than a settled objection.

A fuller set of starting points — including how closely `asinh` resembles `log1p` on non-negative counts, how the transform interacts with the Spotlight objective, and what remains unverified about the inversion itself — is collected in views-models issue #**548**, so that it reaches whoever examines it rather than sitting in a delivery note.

5 Loss Functions

MSE — ordinary squared error, every cell-month weighted equally. The library's own implementation, a thin wrapper around PyTorch's. Used by `dark_river`, `silent_fox` and `red_hawk`.

Spotlight (`SpotlightLossLogcosh`) — in-house, not off the shelf. A three-part objective computed after an `asinh` transform. It scores the *shape* of the error only in cells where the observation or the forecast exceeds one fatality; separately penalises the *level* in cells whose observation exceeds one fatality; and separately penalises forecasting conflict in cells that stayed below it. It uses `log cosh` rather than squared error, so a single very large observation does not dominate the gradient. Used by the other six.

Development and Validation Status

Spotlight is not a one-off function but the production member of a sustained line of work. The first variant was committed in March 2026 and the one used here at the end of April; development continued into late August. The family now comprises six Spotlight variants and three related objectives — roughly 2,250 lines across nine modules — attracting some 216 commits over six months. The library's own documentation sets out five earlier generations of the design, recording for each what it addressed and what it exposed. By any ordinary measure this represents a considerable investment of development effort.

The objective takes a single parameter, and it is derived rather than fitted: the threshold separating event from quiet cells is $\text{asinh}(1) \approx 0.88$, exactly one fatality on the transformed scale.

What the record does not contain is any formal account of it. No citation appears in the source, its documentation, the commit history or the architecture decision records; two other objectives in the same library do cite published work, so the omission is particular to this family rather than a habit of the codebase. Neither is it covered by tests. The library's suite exercises eight of its objectives and this is not among them, so nothing checks its gradients, compares it against a reference implementation, or establishes that the code computes what the written account describes — this for the objective on which 37 of the platform's 42 darts models are trained.

Six months of concentrated development across five successive generations is not the profile of an untried design, and the written account of its construction is detailed. It is reasonable to infer that the objective has been exercised extensively, if informally, during that period. Informal exercise is not validation, however, and the distinction is what this note is obliged to record: what is absent is not the work but the evidence of it. A derivation a third party can follow, and tests that fail when the implementation is wrong, remain outstanding and should be produced for an objective in production use at this scale.

On that basis this note takes no position on whether the objective is sound. It may be excellent, adequate, or quietly wrong, and nothing in the available record distinguishes those possibilities. Six of the nine delivered models were trained on it and three on ordinary squared error, which bears on how much weight a comparison between those two groups can carry.

6 The Delivered Files

One folder per model. Inside each: thirteen Parquet files — a columnar table format that opens in one call from both Python (`pandas.read_parquet`) and R (`arrow::read_parquet`) — named `..._00` through `..._12`, and a short text file recording when the model ran, which version of the software produced it, and which configuration it used.

The thirteen files are thirteen forecast origins, stepping forward one month at a time. An origin is the last month the model was allowed to see; from there it forecasts the next 36 months.

Within every file the earliest month is horizon 1 and the latest horizon 36, so a given target month sits at a different horizon in each file containing it. Metrics computed per origin keep

Table 3: The thirteen forecast origins.

File	Last month seen	Months forecast
`..._00`	December 2017	January 2018 – December 2020
`..._01`	January 2018	February 2018 – January 2021
	...	
`..._12`	December 2018	January 2019 – December 2021

those apart; metrics pooled across origins mix them.

Each file holds 2,333,448 rows — 36 months × 64,818 cells — and five columns.

Table 4: Columns in every file.

Column	Contents
`month_id`	Month
`priogrid_id`	Grid cell
`pred_lr_ged_sb`	Forecast state-based fatalities
`pred_lr_ged_ns`	Forecast non-state fatalities
`pred_lr_ged_os`	Forecast one-sided fatalities

`priogrid_id` **is a cell, not a coordinate.** PRIO-GRID is a 720 × 360 grid of half-degree cells covering the globe, numbered from 1 in the south-west corner and running east before north. To place a row on a map:

Table 5: Converting `priogrid_id` to the centre of its cell.

Quantity	From `priogrid_id`
Row (south to north)	$\texttt{row} = \lfloor(\texttt{priogrid_id} - 1)/720\rfloor$
Column (west to east)	$\texttt{col} = (\texttt{priogrid_id} - 1) \bmod 720$
Latitude	$-90 + (\texttt{row} + 0.5) \times 0.5$
Longitude	$-180 + (\texttt{col} + 0.5) \times 0.5$

In R, for a data frame `d`:

```
d$lat <- -90 + (((d$priogrid_id - 1) %/% 720) + 0.5) * 0.5
d$lon <- -180 + (((d$priogrid_id - 1) %% 720) + 0.5) * 0.5
```

The coordinates are cell *centres*; each cell spans 0.5° in both directions. As a check, `priogrid_id` 148759 — the cell discussed in Section 8 — resolves to 13.25°N, 39.25°E, in Tigray. The 64,818 cells delivered here are the land subset of the 259,200 in the full grid.

Units are fatality counts. A value of 2.4 means 2.4 fatalities; values are continuous rather than integer. The transformations applied during fitting are intended to be inverted before these files are written, so no back-transformation should be applied to them; Section 4 states what is and is not established about that.

Observed values are not included, and are not needed from here. Evaluation joins these forecasts to observed fatalities on `month_id` and `priogrid_id`; the ensemble tool retrieves those itself, and they are otherwise available from the datafactory. The file naming used here already matches what that tool expects.

Nine models × thirteen origins = **117 files, about 2 GB** in total.

7 Not Included in This Delivery

Table 6: Not included in this delivery.

Item	Where it is instead, or why not
Observed values	Fetched by the ensemble tool, or from the datafactory (Section 6)
Evaluation metrics	None computed — see Section 8
Training logs	Not shipped. `calibration_log.txt`, which the ensemble tool can optionally read to resolve run timestamps, is among them; timestamps can be passed explicitly instead
Other partitions	Calibration only — no validation partition, no true forecasts
Model artefacts	No weights, no checkpoints

8 Caveats and Notes

No model here has a predictive distribution. These are point forecasts — one number per cell-month, no draws, no quantiles, no intervals. All nine are configured as deterministic predictors, so a different export would not recover a distribution; it was never produced. Nothing that scores a *single model's* uncertainty — CRPS of its predictive density, interval coverage, density calibration — can be computed from these files.

That is a limitation of this delivery against the direction the VIEWS Prediction Challenge sets out (Hegre et al., 2025), which argues that for a field this skewed a distribution is closer to what is needed than a point estimate. These nine do not provide one.

One qualification, since it would otherwise look like a contradiction in the output: the ensemble tool can still produce a CRPS. It does so by treating the member models' point forecasts as an empirical sample and scoring the spread across models. That is a property of the ensemble, not of any model in it, and it is not the same quantity as a CRPS computed from a model's own posterior. Worth knowing before the two are compared.

Nothing here has been evaluated. I checked only that the files are not broken: no negatives, no missing values, no duplicate cell-months, the expected row counts, and totals within the same order of magnitude as observed. That rules out a failed back-transformation or a collapsed model and establishes nothing else.

It is nine models, not eleven. Two others were set up to draw 100 samples per forecast,

making them probabilistic counterparts of `dancing_monkey` and `dark_necessities`. A point delivery needs one value per cell, and at one sample their configurations are byte-identical to those siblings — the same model twice. Running them as intended is blocked by memory: 100 samples for 64,818 cells across thirteen origins needs far more RAM than any machine we have.

One observation will dominate any squared-error metric. December 2021, cell 148759 has **113,395** observed fatalities; the next largest anywhere in the window is 5,395. That month falls in only one of the thirteen origins (`..._12`), where that single cell is about 41% of observed fatalities. I do not know whether it is a real event or a data error. Either way it is better settled before the numbers are produced than explained afterwards.

The software is pinned; the data is not. The title page records the three versions that determine the output — the model library, the framework and the data backend — along with the `views-models` commit the configurations were read at. Between them those fix the code. They do not fix the *input*: the data backend served this run from a live store with no snapshot identifier, and the provenance record written beside the fetched data shows drift detection did not run. So re-running these nine models at these versions would reproduce the method, and would not guarantee reproducing the numbers, because the underlying observations may have been revised in the meantime. Worth knowing before anyone treats a re-run as a check.

A correction to my September note. It said the full predictive distributions had been retained, so alternative summaries could be produced from those models without re-running them. They were not kept.

9 Next Steps

The delivery is complete and the files are ready to be evaluated. Three things would help, and each is quick.

1. **Which column naming should the evaluation use?** These files use `pred_lr_ged_sb` and its two siblings; the September delivery used `pred_lr_sb_best`. Renaming either set takes minutes and needs no re-run, but somebody has to say which convention the evaluation code expects.

2. **Which of the three September variants has the evaluation been built on?** You were sent a mean, a 95th percentile and a conditional mean, each computed from that model family's predictive distribution. These nine have no distribution, so only the mean has an equivalent here. If the work rests on one of the other two, the two deliveries are not comparable and it is better to know before they are placed side by side.

3. **Can someone establish whether December 2021, cell 148759 is a real event or a data error?** At 113,395 fatalities against a next-largest of 5,395 it will dominate any squared-error metric computed over the final origin, and the answer changes how that origin should be handled.

Two further items are open on our side rather than yours. The transform these models were

fitted on has never been examined; the starting points are collected in views-models issue #548. And the objective six of the nine were trained on has no tests and no derivation, as Section 5 sets out; producing those sits with its author.

Questions about the data, the models or anything in this note come to me.

References

Bellemare, M. F. and Wichman, C. J. (2020). Elasticities and the inverse hyperbolic sine transformation. *Oxford Bulletin of Economics and Statistics*, 82(1):50–61.

Chen, J. and Roth, J. (2024). Logs with zeros? some problems and solutions. *The Quarterly Journal of Economics*, 139(2):891–936.

Chen, S.-A., Li, C.-L., Yoder, N., Arik, S. O., and Pfister, T. (2023). TSMixer: An all-MLP architecture for time series forecasting. *Transactions on Machine Learning Research*. arXiv:2303.06053.

Das, A., Kong, W., Leach, A., Mathur, S., Sen, R., and Yu, R. (2023). Long-term forecasting with TiDE: Time-series dense encoder. *arXiv preprint arXiv:2304.08424*.

Duan, N. (1983). Smearing estimate: A nonparametric retransformation method. *Journal of the American Statistical Association*, 78(383):605–610.

Hegre, H., Vesco, P., Colaresi, M., et al. (2025). The 2023/24 VIEWS prediction challenge: Predicting the number of fatalities in armed conflict, with uncertainty. *Journal of Peace Research*. doi:10.1177/00223433241300862.

Kim, T., Kim, J., Tae, Y., Park, C., Choi, J.-H., and Choo, J. (2022). Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations (ICLR)*.

Oreshkin, B. N., Carpov, D., Chapados, N., and Bengio, Y. (2020). N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*. arXiv:1905.10437.